

CS 550

Formal Verification

Midterm Exam

November 27, 2025

Name: Ada Lovelace
Sciper: 111111

Room: GR A3 30
Seat: 314

1. You have **165 minutes** to complete the exam.
2. Place your CAMIPRO card on your desk.
3. Put all electronic devices in a bag away from the bench.
4. Write your final answers using a permanent pen (no pencils, no erasable pens).
5. This exam is 19 pages long, including this cover page. Check that you have all the pages.
6. The exercises are not ordered by how difficult they may be. If you are stuck on an exercise, you can skip it and come back to it later.
7. Answer all questions in the provided space. Do not submit additional sheets unless you get explicit permission to do so. Do not unstaple the given sheets. Material in the scratch area will not be graded.
8. The maximum number of points on the exam is **40**.

Question:	1	2	3	4	5	6	7	Total
Points:	3	5	5	6	4	6	11	40
Score:								



Page intentionally left blank.

**Question 1.**

(3 points)

Consider the following propositional formula α :

$$((p \implies q) \wedge (p \implies \neg q)) \implies \neg p .$$

1. 1 point Transform α to an equivalent formula in negation-normal form that uses only $\wedge$, $\vee$, and $\neg$. Use a polynomial-time algorithm. Denote the result by ϕ .
2. 1 point Draw an abstract syntax tree for ϕ , with propositional connectives as the internal nodes. Label the binary nodes in your tree such that children have larger numbers than their parent, starting from 0 at the root (you can, for example use depth-first tree traversal).
3. 1 point Convert the formula into an equisatisfiable clausal normal form (CNF) by applying Tseitin's transformation. For variables you introduce for binary connectives, use x_n as the name, where n is the node number for the corresponding subformula in your syntax tree.

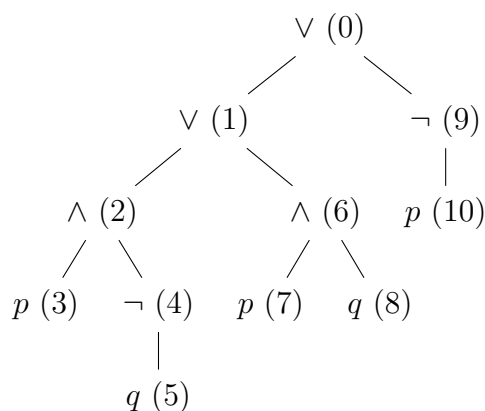
Solution:

1. The negation-normal form transformation of α is:

$$\begin{aligned}
 \alpha &= ((p \implies q) \wedge (p \implies \neg q)) \implies \neg p \\
 &\iff \neg((p \implies q) \wedge (p \implies \neg q)) \vee \neg p \\
 &\iff (\neg(p \implies q) \vee \neg(p \implies \neg q)) \vee \neg p \\
 &\iff ((p \wedge \neg q) \vee (p \wedge q)) \vee \neg p .
 \end{aligned}$$

The last line is the required negation-normal form ϕ .

2. The abstract syntax tree for ϕ is (we label *all* nodes, depth-first, but this is not necessary):





3. The formula ϕ after introducing new variables for (non-atomic) subformulas is:

$$\begin{aligned} & x_0 \\ & \wedge x_0 \leftrightarrow (x_1 \vee \neg p) \\ & \wedge x_1 \leftrightarrow (x_2 \vee x_6) \\ & \wedge x_2 \leftrightarrow (p \wedge \neg q) \\ & \wedge x_6 \leftrightarrow (p \wedge q) . \end{aligned}$$

Finally, the clausal normal form for ϕ is:

$$\begin{aligned} & x_0 \\ & \wedge (\neg x_0 \vee x_1 \vee \neg p) \wedge (\neg x_1 \vee x_0) \wedge (\neg p \vee x_0) \\ & \wedge (\neg x_1 \vee x_2 \vee x_6) \wedge (\neg x_2 \vee x_1) \wedge (\neg x_6 \vee x_1) \\ & \wedge (\neg x_2 \vee p) \wedge (\neg x_2 \vee \neg q) \wedge (\neg p \vee q \vee x_2) \\ & \wedge (\neg x_6 \vee p) \wedge (\neg x_6 \vee q) \wedge (\neg p \vee \neg q \vee x_6) . \end{aligned}$$

**Question 2.**

(5 points)

Consider a first-order theory with two binary relations, L (meant to denote some partial ordering relation $\sqsubseteq$) and E (meant to denote equality $=$). We define the shorthand $\text{lub}(x, y, z)$ as

$$L(x, z) \wedge L(y, z) \wedge \forall w. (L(x, w) \wedge L(y, w)) \implies L(z, w) .$$

Then, define the axiom A_{lub} as

$$\forall x, y. \exists z. \text{lub}(x, y, z) .$$

1. 1 point Show the result of transforming the axiom A_{lub} into an equisatisfiable finite set of clauses.
2. 4 points Given the set of numbered clausal form axioms below, and the axiom A_{lub} , prove the following statement by constructing a resolution refutation for its negation:

$$\phi := \forall x, y, z_1, z_2. (\text{lub}(x, y, z_1) \wedge \text{lub}(x, y, z_2)) \implies E(z_1, z_2) .$$

Number each step following the axioms and use them to refer to premises of each step in your proof.

Note that $\text{lub}(x, y, z)$ is just shorthand, and you can expand it as needed.

Start your proof with the following axioms for equality E and partial order L :

- 1 : $E(x, x)$
- 2 : $\neg E(x, y) \vee \neg E(y, z) \vee E(x, z)$
- 3 : $E(x, y) \vee \neg E(y, x)$
- 4 : $L(x, x)$
- 5 : $\neg L(x, y) \vee \neg L(y, z) \vee L(x, z)$
- 6 : $\neg L(x, y) \vee \neg L(y, x) \vee E(x, y)$

where x, y , and z are free variables (implicitly universally quantified). Add clauses for A_{lub} and proceed with resolution steps.

Solution:

1. Transforming A_{lub} into clausal normal form. After transforming to negation normal form and applying Skolemization, we have:

$$\forall x, y. L(x, f(x, y)) \wedge L(y, f(x, y)) \wedge \forall w. \neg L(x, w) \vee \neg L(y, w) \vee L(f(x, y), w)$$

where f of arity 2 is a Skolem function, and prenexifying:

$$\forall x, y, w. L(x, f(x, y)) \wedge L(y, f(x, y)) \wedge (\neg L(x, w) \vee \neg L(y, w) \vee L(f(x, y), w))$$



resulting in the clauses:

$$7 : L(x, f(x, y))$$

$$8 : L(y, f(x, y))$$

$$9 : \neg L(x, w) \vee \neg L(y, w) \vee L(f(x, y), w)$$

where w is additionally a free variable.

2. First, we will negate and clausify ϕ . Then, we can attempt to derive a contradiction using resolution. Negating the statement gives:

$$\exists x, y, z_1, z_2. \text{lub}(x, y, z_1) \wedge \text{lub}(x, y, z_2) \wedge \neg E(z_1, z_2) .$$

and after Skolemization:

$$\text{lub}(a, b, c_1) \wedge \text{lub}(a, b, c_2) \wedge \neg E(c_1, c_2)$$

where a, b, c_1 , and c_2 are fresh constants. Expanding lub :

$$\begin{aligned} & L(a, c_1) \\ & \wedge L(b, c_1) \\ & \wedge \forall w_1. (L(a, w_1) \wedge L(b, w_1)) \implies L(c_1, w_1) \\ & \wedge L(a, c_2) \wedge L(b, c_2) \\ & \wedge \forall w_2. (L(a, w_2) \wedge L(b, w_2)) \implies L(c_2, w_2) \\ & \wedge \neg E(c_1, c_2) \end{aligned}$$

This results in the clauses:

$$\begin{aligned} 10 & : L(a, c_1) \\ 11 & : L(b, c_1) \\ 12 & : \neg L(a, w) \vee \neg L(b, w) \vee L(c_1, w) \\ 13 & : L(a, c_2) \\ 14 & : L(b, c_2) \\ 15 & : \neg L(a, w) \vee \neg L(b, w) \vee L(c_2, w) \\ 16 & : \neg E(c_1, c_2) \end{aligned}$$

We can now write a resolution refutation using the clauses above:

$$\begin{aligned} 17 & : \neg L(b, c_2) \vee L(c_1, c_2) & (13, 12, w := c_2) \\ 18 & : L(c_1, c_2) & (14, 17) \\ 19 & : \neg L(b, c_1) \vee L(c_2, c_1) & (10, 15, w := c_1) \\ 20 & : L(c_2, c_1) & (11, 19) \\ 21 & : \neg L(c_2, c_1) \vee E(c_1, c_2) & (18, 6, x := c_1, y := c_2) \\ 22 & : E(c_1, c_2) & (20, 21) \\ 23 & : \perp & (22, 16) \end{aligned}$$

Since we have derived a contradiction from the negation of ϕ , we can conclude that ϕ is valid.

**Question 3.**

(5 points)

Consider the following Presburger arithmetic formula ϕ :

$$\forall x. \exists y. x < y \wedge y < x + 6 \wedge 3 \mid (y + 2)$$

Check whether the formula is true or false using quantifier elimination. Show your steps. You may use arbitrary equivalence-preserving transformations on quantifier-free subformulas in a single step.

Solution:

1. We will compute a formula ϕ' equivalent to ϕ which is quantifier-free, and show that $\phi' \iff \top$. We have

$$\begin{aligned}
 & \forall x. \exists y. x < y \wedge y < x + 6 \wedge 3 \mid (y + 2) \\
 \iff & \forall x. \bigvee_{i=1}^3 (x < x + i \wedge x + i < x + 6 \wedge 3 \mid (x + i + 2)) && \text{(eliminate } \exists y) \\
 \iff & \forall x. \bigvee_{i=1}^3 (x + i < x + 6 \wedge 3 \mid (x + i + 2)) && (x < x + i \equiv \top) \\
 \iff & \forall x. \bigvee_{i=1}^3 (3 \mid (x + i + 2)) && (x + i < x + 6 \equiv \top, i \in \{1, 2, 3\}) \\
 \iff & \neg \exists x. \neg \bigvee_{i=1}^3 (3 \mid (x + i + 2)) && (\forall \rightarrow \neg \exists \neg) \\
 \iff & \neg \exists x. \bigwedge_{i=1}^3 \neg (3 \mid (x + i + 2)) && \text{(NNF)} \\
 \iff & \neg \exists x. \bigwedge_{i=1}^3 \bigvee_{j=1}^2 (3 \mid (x + i + j + 2)) && \text{(unfolding } \neg 3 \mid \dots) \\
 \iff & \neg \exists x. \bigwedge_{i=1}^3 \bigvee_{j=3}^4 (3 \mid (x + i + j)) && \text{(renaming } j + 2 \rightarrow j) \\
 \iff & \neg \exists x. \bigvee_{j_1, j_2, j_3 \in \{3, 4\}} \bigwedge_{i=1}^3 (3 \mid (x + i + j_i)) && \text{(distributing } \wedge \text{ over } \vee) \\
 \iff & \neg \bigvee_{k=1}^3 \bigvee_{j_1, j_2, j_3 \in \{3, 4\}} \bigwedge_{i=1}^3 (3 \mid (k + i + j_i)) && \text{(eliminate } \exists x) \\
 \iff & \neg \bigvee_{k=1}^3 \bigvee_{j_1, j_2, j_3 \in \{3, 4\}} (3 \mid (k + j_1 + 1)) \wedge (3 \mid (k + j_2 + 2)) \wedge (3 \mid (k + j_3 + 3)) \\
 \iff & \neg \perp \iff \top && \text{(evaluation)}
 \end{aligned}$$

This is ϕ' , equivalent to ϕ , as desired, and clearly $\phi' \iff \top$, so $\phi \iff \top$.


Question 4. (6 points)

Let $(C, \leq)$ and $(A, \sqsubseteq)$ be two partial orders. Let $F : C \rightarrow A$ and $G : A \rightarrow C$ be two functions such that (F, G) they satisfy the Galois condition:

$$F(c) \sqsubseteq a \iff c \leq G(a).$$

1. 2 points Prove that F and G are monotone functions, and thus that (F, G) is a Galois connection.
2. 4 points Suppose there exists another monotone function $H : A \rightarrow C$ such that (F, H) is also a Galois connection. Then, prove that $G(a) = H(a)$ for every $a \in A$.

For the second part, you may assume the results of the first.

Solution:

1. **Monotonicity.** We will show that F is monotone; the proof for G is similar. Let $c_1, c_2 \in C$ such that $c_1 \leq c_2$. We need to show that $F(c_1) \sqsubseteq F(c_2)$. We have

$$\begin{aligned}
 & F(c_2) \sqsubseteq F(c_2) && \text{(reflexivity of } \sqsubseteq \text{)} \\
 \iff & c_2 \leq G(F(c_2)) && \text{(Galois condition)} \\
 \implies & c_1 \leq G(F(c_2)) && \text{(transitivity with } c_1 \leq c_2 \text{)} \\
 \iff & F(c_1) \sqsubseteq F(c_2) . && \text{(Galois condition)}
 \end{aligned}$$

2. **Uniqueness.** Pick an arbitrary $a \in A$. We will show that $G(a) = H(a)$. By the definition of Galois connection applied to pairs (F, H) and (F, G) , we have for every $c \in C$:

$$c \leq H(a) \iff F(c) \sqsubseteq a \iff c \leq G(a)$$

In particular, choosing $c = G(a)$, we get:

$$G(a) \leq H(a) \iff G(a) \leq G(a) \tag{1}$$

and similarly, choosing $c = H(a)$, we get:

$$H(a) \leq G(a) \iff H(a) \leq H(a) \tag{2}$$

Since both $G(a) \leq G(a)$ and $H(a) \leq H(a)$ are true by reflexivity of $\leq$, we conclude from (1) and (2) that both $G(a) \leq H(a)$ and $H(a) \leq G(a)$ hold. By antisymmetry of $\leq$, we thus have $G(a) = H(a)$, as desired.

**Question 5.**

(4 points)

Let $(A, \leq)$ be a partial order. For each $a \in A$, define the set L_a as

$$L_a = \{x \in A \mid x \leq a\}.$$

Let $C = \{L_a \mid a \in A\}$ be the collection of all such sets. Define the function $L : A \rightarrow C$ by $L(a) = L_a$.

Consider the partial order $(C, \subseteq)$, where $\subseteq$ is the usual subset relation. Answer each of the following questions with a proof or a counterexample:

1. 1 point Is L monotone from $(A, \leq)$ to $(C, \subseteq)$?
2. 1 point Is L injective?
3. 1 point Does there exist a function $K_1 : C \rightarrow A$ such that (L, K_1) is a Galois connection between $(A, \leq)$ and $(C, \subseteq)$?
4. 1 point Does there exist a function $K_2 : C \rightarrow A$ such that (L, K_2) is a Galois insertion of $(C, \subseteq)$ into $(A, \leq)$?

Recall that (L, K) is a Galois insertion of $(C, \subseteq)$ into $(A, \leq)$ if it is a Galois connection between them and for every $c \in C$, we have $L(K(c)) = c$.

Solution:

1. **Yes**, L is monotone. For any $a_1, a_2 \in A$ such that $a_1 \leq a_2$, we have for any $x \in A$:

$$\begin{aligned}
 x &\in L(a_1) \\
 \iff x &\in A \wedge x \leq a_1 && \text{(definition of } L_{a_1}) \\
 \implies x &\in A \wedge x \leq a_2 && \text{(transitivity with } a_1 \leq a_2) \\
 \iff x &\in L(a_2) && \text{(definition of } L_{a_2})
 \end{aligned}$$

and thus $L(a_1) \subseteq L(a_2)$.

2. **Yes**, L is injective. Suppose $L(a_1) = L(a_2)$ for some $a_1, a_2 \in A$. By definition of L , we have $a_1 \in L(a_1)$ and thus $a_1 \in L(a_2)$. Again by definition of L , this implies that $a_1 \leq a_2$. Similarly, by swapping a_1 and a_2 , we get $a_2 \leq a_1$. By antisymmetry of $\leq$, we have that $a_1 = a_2$. Thus, L is injective.
3. **Yes**, such a function K_1 exists. Note that for every $c \in C$, there exists some $a \in A$ such that $c = L(a)$ by definition of L and C . Thus, L is surjective. From 2 we know that L is also injective, and thus L is bijective. Therefore, there exists a function $L^{-1} : C \rightarrow A$ which is the inverse of L . We can then define K_1 as $K_1 = L^{-1}$.

We need to show that (L, K_1) is a Galois connection. We need to show that K_1 is monotone, and that for every $a \in A$ and $c \in C$

$$L(a) \subseteq c \iff a \leq K_1(c).$$



- K_1 is monotone. Consider any $c_1, c_2 \in C$ such that $c_1 \subseteq c_2$. Define $a_1 = K_1(c_1)$ and $a_2 = K_1(c_2)$. By the definition of K_1, L^{-1} , and L , we have $L_{a_1} = c_1$ and $L_{a_2} = c_2$. Now, we have

$$\begin{aligned}
& c_1 \subseteq c_2 \\
& \iff L_{a_1} \subseteq L_{a_2} \\
& \iff \{x \in A \mid x \leq a_1\} \subseteq \{x \in A \mid x \leq a_2\} \\
& \iff \forall x \in A. x \leq a_1 \implies x \leq a_2 \\
& \implies a_1 \leq a_1 \implies a_1 \leq a_2 \quad (\text{instantiating } x := a_1) \\
& \iff a_1 \leq a_2 \quad (\text{reflexivity of } \leq) \\
& \iff K_1(c_1) \leq K_1(c_2) . \quad (\text{definition of } a_1, a_2)
\end{aligned}$$

- $L(a) \subseteq c \implies a \leq K_1(c)$. We have

$$\begin{aligned}
& L(a) \subseteq c \\
& \implies K_1(L(a)) \leq K_1(c) \quad (\text{monotonicity of } K_1) \\
& \iff L^{-1}(L(a)) \leq K_1(c) \quad (\text{definition of } K_1) \\
& \iff a \leq K_1(c) . \quad (\text{definition of } L^{-1})
\end{aligned}$$

- $L(a) \subseteq c \iff a \leq K_1(c)$. We have

$$\begin{aligned}
& a \leq K_1(c) \\
& \implies L(a) \subseteq L(K_1(c)) \quad (\text{monotonicity of } L) \\
& \iff L(a) \subseteq L(L^{-1}(c)) \quad (\text{definition of } K_1) \\
& \iff L(a) \subseteq c . \quad (\text{definition of } L^{-1})
\end{aligned}$$

Thus, K_1 is monotone, and (L, K_1) is a Galois connection between $(A, \leq)$ and $(C, \subseteq)$.

4. **Yes**, we can choose $K_2 = K_1 = L^{-1}$ as defined in 3. For every $c \in C$, we have

$$L(K_2(c)) = L(L^{-1}(c)) = c$$

by the definition of L^{-1} . Thus, (L, K_2) is a Galois insertion.

**Question 6.**

(6 points)

Prove the following first-order logic sequent using Lisa proof steps:

$$\forall x. P(x) \implies S(x), \exists x. \neg S(x) \quad \vdash \quad \exists x. \neg P(x)$$

Exact syntax of proof steps is not important. You may begin with the following definitions and theorem scaffold.

Common proof steps you may find useful are described in the appendix.

Solution:

```

val x = variable[Ind]
val P = variable[Ind >>: Prop]
val S = variable[Ind >>: Prop]

Theorem( ( ∀(x, P(x) ⇒ S(x)), ∃(x, !S(x)) ) ⊢ ∃(x, !P(x)) ):
  // your proof here
  have( ∀(x, P(x) ⇒ S(x)) ⊢ P(x) ⇒ S(x) ) by InstantiateForall

  have( ∀((x, P(x) ⇒ S(x)), !S(x)) ⊢ !P(x) ) by Tautology.from(lastStep)

  // or simpler instead of Tautology, by Weakening:
  // thenHave( ∀((x, P(x) ⇒ S(x)), !S(x)) ⊢ P(x) ⇒ S(x) ) by Weakening
  // thenHave( ∀((x, P(x) ⇒ S(x)), !S(x)) ⊢ !P(x) ) by Weakening

  thenHave( ∀((x, P(x) ⇒ S(x)), !S(x)) ⊢ ∃(x, !P(x)) ) by RightExists
  thenHave( ∀((x, P(x) ⇒ S(x)), ∃(x, !S(x))) ⊢ ∃(x, !P(x)) ) by LeftExists

```

**Question 7.**

(11 points)

Consider the following program over mathematical (unbounded) integers (BigInts):

```
def myLittleProgram(): BigInt =
  var a: BigInt = havoc()
  var b: BigInt = havoc() ①

  assume(a ≥ 0 ∧ b ≥ 0) ①

  while ② (a != 0 ∧ b != 0) { ③
    if a ≥ b then ④
      a = a - b ⑤
    else ⑥
      b = b - a ⑦
  } ⑧

  assert(a ≥ 0 ∧ b ≥ 0) ⑨
  max(a, b)
```

We identify program points using numbers ①, ②, ③, ... Recall that `havoc()` non-deterministically returns an arbitrary integer value, `assume(p)` restricts the execution to states where p holds.

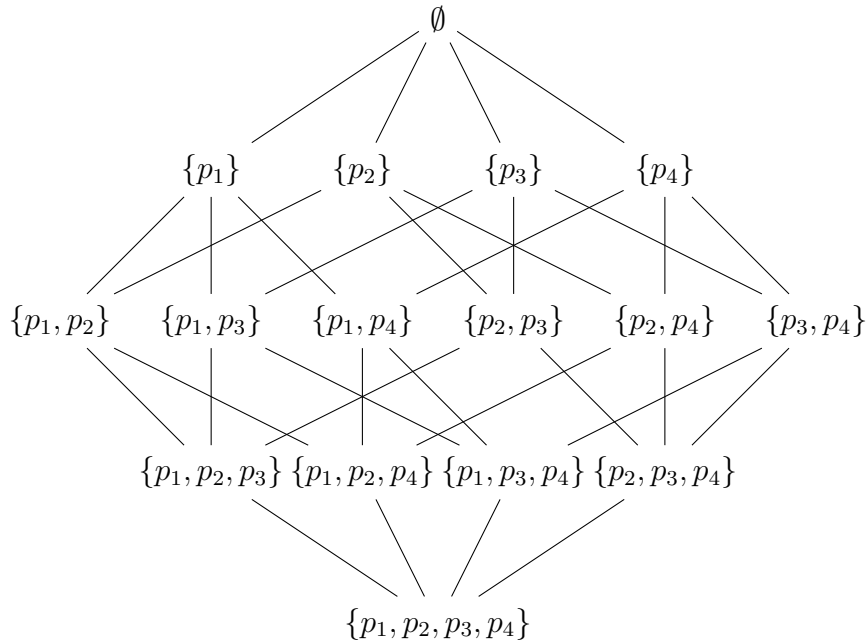
`assert(p)` checks that p holds in the state, or passes the control-flow to a distinct error state.

Consider the following set of predicates, labelled p_1, p_2, p_3 , and p_4 from left to right:

$$\mathcal{P} = \{a \geq 0, b \geq 0, a \geq b, a > 0\}$$

$p_1 \quad p_2 \quad p_3 \quad p_4$

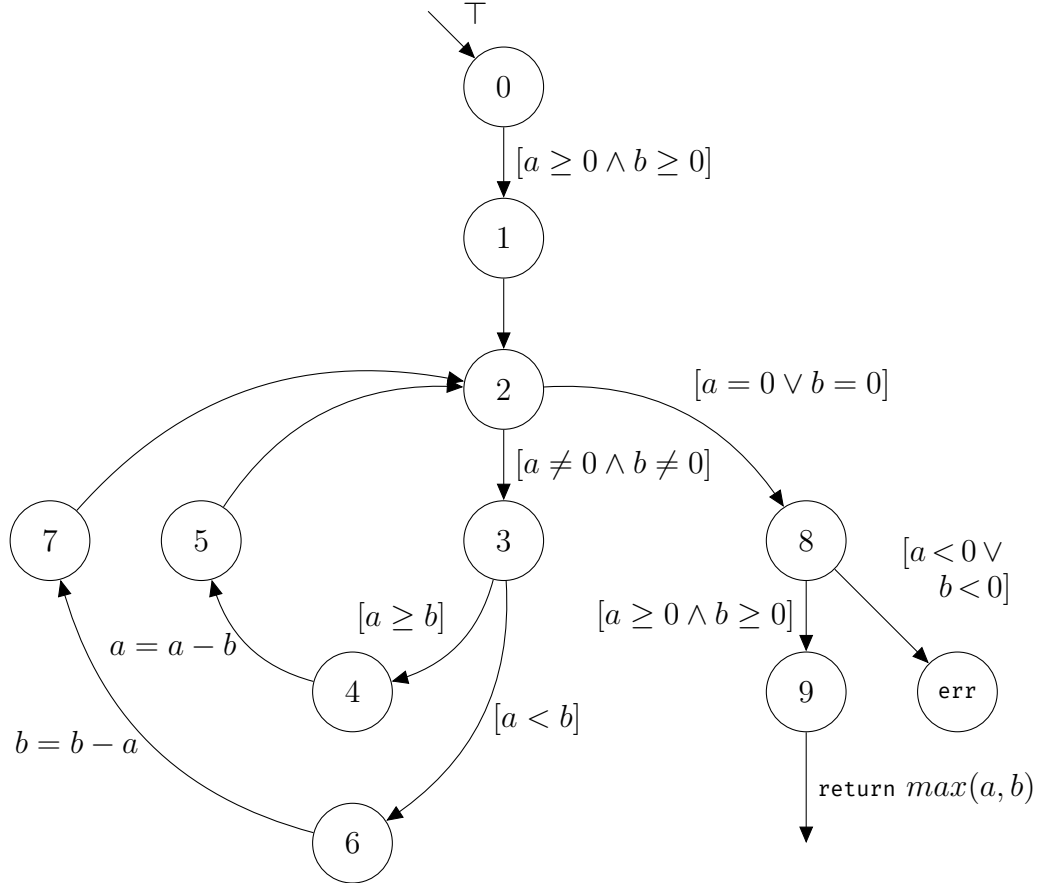
1. 1 point Draw the Hasse diagram for the lattice $(2^{\mathcal{P}}, \sqsubseteq)$ of conjuncts over elements of $\mathcal{P}$ ordered by the predicate abstraction lattice order.

Solution:



2. [2 points] Using the numbered program points above as the set of states, draw a control-flow graph representing `myLittleProgram`. Annotate the edges with guards and assignments. Introduce a program point `err` to which the program moves when the assertion condition does not hold. There should be no outgoing edges from `err`. The set of states is $Q = \{0, 1, \dots, 9, \text{err}\}$.

Solution:



3. [2 points] Represent the semantics of each edge in your transition system as a formula over pre- and post-state variables, (a, b) and (a', b') . Represent the edge from state $i \in Q$ to $j \in Q$ as the formula labelled $R_{i \rightarrow j}(a, b, a', b')$.

Solution: We will represent a relation for an edge from state i to j as $R_{i \rightarrow j}(a, b, a', b')$. The “initial” edge coming into 0 and the “final” edge exiting from 9 are not included.

$$R_{0 \rightarrow 1}(a, b, a', b') = (a' \geq 0) \wedge (b' \geq 0)$$

$$R_{1 \rightarrow 2}(a, b, a', b') = (a' = a) \wedge (b' = b)$$

$$R_{2 \rightarrow 3}(a, b, a', b') = (a \neq 0) \wedge (b \neq 0) \wedge (a' = a) \wedge (b' = b)$$

$$R_{3 \rightarrow 4}(a, b, a', b') = (a \geq b) \wedge (a' = a) \wedge (b' = b)$$

$$R_{4 \rightarrow 5}(a, b, a', b') = (a' = a - b) \wedge (b' = b)$$



$$\begin{aligned}
R_{5 \rightarrow 2}(a, b, a', b') &= (a' = a) \wedge (b' = b) \\
R_{3 \rightarrow 6}(a, b, a', b') &= (a < b) \wedge (a' = a) \wedge (b' = b) \\
R_{6 \rightarrow 7}(a, b, a', b') &= (a' = a) \wedge (b' = b - a) \\
R_{7 \rightarrow 2}(a, b, a', b') &= (a' = a) \wedge (b' = b) \\
R_{2 \rightarrow 8}(a, b, a', b') &= (a = 0 \vee b = 0) \wedge (a' = a) \wedge (b' = b) \\
R_{8 \rightarrow 9}(a, b, a', b') &= (a \geq 0) \wedge (b \geq 0) \wedge (a' = a) \wedge (b' = b) \\
R_{8 \rightarrow \text{err}}(a, b, a', b') &= ((a < 0) \vee (b < 0)) \wedge (a' = a) \wedge (b' = b)
\end{aligned}$$

4. 1 point Denote the abstract set of values at each program point i by the function $S^\#(i)$ for $i \in \{0, 1, \dots, 9, \text{err}\}$. Further, denote the abstract transfer function by $T^\#(S, R)$. It takes $S \in 2^{\mathcal{P}}$, an abstract set of values, and R , a formula relating concrete states as in the previous part. It computes the new abstract set of values $\in 2^{\mathcal{P}}$.

Using $S^\#$ and $T^\#$, write a set of recursive equations modelling the transition system, whose least fixed point solution gives the abstract set of values at each program point.

Solution: Note that $\top$ here is as it is in the abstract lattice, the empty conjunction.

$$\begin{aligned}
S^\#(0) &= \top \\
S^\#(1) &= T^\#(S^\#(0), R_{0 \rightarrow 1}) \\
S^\#(2) &= T^\#(S^\#(1), R_{1 \rightarrow 2}) \sqcup T^\#(S^\#(5), R_{5 \rightarrow 2}) \sqcup T^\#(S^\#(7), R_{7 \rightarrow 2}) \\
S^\#(3) &= T^\#(S^\#(2), R_{2 \rightarrow 3}) \\
S^\#(4) &= T^\#(S^\#(3), R_{3 \rightarrow 4}) \\
S^\#(5) &= T^\#(S^\#(4), R_{4 \rightarrow 5}) \\
S^\#(6) &= T^\#(S^\#(3), R_{3 \rightarrow 6}) \\
S^\#(7) &= T^\#(S^\#(6), R_{6 \rightarrow 7}) \\
S^\#(8) &= T^\#(S^\#(2), R_{2 \rightarrow 8}) \\
S^\#(9) &= T^\#(S^\#(8), R_{8 \rightarrow 9}) \\
S^\#(\text{err}) &= T^\#(S^\#(8), R_{8 \rightarrow \text{err}})
\end{aligned}$$



5. 3 points Compute and show the least fixed point of the recursive equations computed in the previous part. You do not need to prove that the result is indeed a fixed point or the least.

Solution:

We will compute the least fixed point by iteratively computing the values of $S^\#(i)$, starting from the piecewise least abstract element $(\perp, \perp, \dots, \perp)$. In each iteration, we will compute the next set of values by substituting the current values into the right-hand sides of the equations.

We will only write the values that change in each iteration.

- Iteration 0: initial state; $S^\#(i) = \perp$ for each state i .
- Iteration 1: $S^\#(0) = \top$.
- Iteration 2: $S^\#(1) = a \geq 0 \wedge b \geq 0$.
- Iteration 3: $S^\#(2) = a \geq 0 \wedge b \geq 0$.
- Iteration 4: $S^\#(3) = a > 0 \wedge a \geq 0 \wedge b \geq 0$ and $S^\#(8) = a \geq 0 \wedge b \geq 0$. Note that our predicate abstraction does not contain $b > 0$.
- Iteration 5: $S^\#(4) = a > 0 \wedge a \geq 0 \wedge a \geq b \wedge b \geq 0$, $S^\#(6) = a > 0 \wedge a \geq 0 \wedge b \geq 0$, and $S^\#(9) = a \geq 0 \wedge b \geq 0$.
- Iteration 6: $S^\#(5) = a \geq 0$ and $S^\#(7) = a > 0 \wedge a \geq 0$.
- Iteration 7: $S^\#(2) = a \geq 0$.
- Iteration 8: $S^\#(3) = a > 0 \wedge a \geq 0$ and $S^\#(8) = a \geq 0$.
- Iteration 9: $S^\#(4) = a > 0 \wedge a \geq 0 \wedge a \geq b$ and $S^\#(\text{err}) = a \geq 0 \wedge a \geq b$.
- Iteration 10: $S^\#(5) = a \geq 0$.

Nothing changes in the next iteration, so we have reached a fixed point. The fact that this is the least fixed point follows from monotonicity and the iteration argument seen in the lectures.



The final values are:

$$\begin{aligned}
 S^\#(0) &= \top \\
 S^\#(1) &= a \geq 0 \wedge b \geq 0 \\
 S^\#(2) &= a \geq 0 \\
 S^\#(3) &= a > 0 \wedge a \geq 0 \\
 S^\#(4) &= a > 0 \wedge a \geq 0 \wedge a \geq b \\
 S^\#(5) &= a \geq 0 \\
 S^\#(6) &= a > 0 \wedge a \geq 0 \wedge b \geq 0 \\
 S^\#(7) &= a > 0 \wedge a \geq 0 \\
 S^\#(8) &= a \geq 0 \\
 S^\#(9) &= a \geq 0 \wedge b \geq 0 \\
 S^\#(\text{err}) &= a \geq 0 \wedge a \geq b
 \end{aligned}$$

6. 1 point Give a mathematical expression describing the set of possible concrete values of a and b corresponding to the computed fixed point at the program point `err`?

1 point Is there an execution of this program that reaches the `err` state? Justify your answer.

Solution:

Since $S^\#(\text{err}) = a \geq 0 \wedge a \geq b$, the corresponding set of concrete states $S(\text{err})$ is $\{(a, b) \mid a \geq 0 \wedge a \geq b\}$.

Choose a value from this set, say $(a, b) = (1, 0)$. On running the program with these values, we see that the assertion passes and the program does not actually reach the `err` state. The same is true for every value in this set. This can be shown with a more precise predicate abstraction, by adding $b > 0$ to $\mathcal{P}$, for instance.

This entails that the current predicate abstraction is not precise enough to prove the safety of the program and the current analysis is inconclusive.



Appendix

Basic Lisa proof steps for quantifiers and propositional reasoning, with usage examples.

Tautology. Proves a propositional tautology.

$$\frac{\Gamma_1 \vdash \Phi_1 \quad \Gamma_2 \vdash \Phi_2 \quad \dots \quad \Gamma_n \vdash \Phi_n}{\Gamma \vdash \Psi} \text{Tautology}$$

such that $(\bigwedge_{i=1}^n (\Gamma_i \implies \Phi_i)) \implies (\Gamma \implies \Psi)$ is provable in propositional logic.

Usage example:

```
have(B ⊢ A ⟹ B) by Tautology
have((A, B) ⊢ B ⟹ A) by Tautology.from(lastStep)
```

InstantiateForall. Instantiate a universally quantified formula.

$$\frac{}{\Gamma, \forall x. P(x) \vdash P(t)} \text{InstantiateForall (1)}$$

$$\frac{\Gamma \vdash \forall x. P(x)}{\Gamma \vdash P(t)} \text{InstantiateForall (2)}$$

where t is any term.

Usage example:

```
// version 1
have((B, ∀(x, P(x))) ⊢ P(t)) by InstantiateForall

// version 2
have(B ⊢ ∀(x, P(x))) by ...
thenHave(B ⊢ P(y)) by InstantiateForall(y)
```

RightExists. Introduce an existential quantifier on the right.

$$\frac{\Gamma \vdash P(t)}{\Gamma \vdash \exists x. P(x)} \text{RightExists}$$

where t is any term.

Usage example:

```
have(B ⊢ P(t)) by ...
thenHave(B ⊢ ∃(x, P(x))) by RightExists
```

LeftExists. Introduce an existential quantifier on the left.

$$\frac{\Gamma, P(x) \vdash \Phi}{\Gamma, \exists x. P(x) \vdash \Phi} \text{LeftExists}$$

where x is *not free* in the resulting sequent.

Usage example:

```
have((B, P(y)) ⊢ Q) by ...
thenHave((B, ∃(x, P(x))) ⊢ Q) by LeftExists
```



RightForall. Introduce a universal quantifier on the right.

$$\frac{\Gamma \vdash P(x)}{\Gamma \vdash \forall x. P(x)} \text{RightForall}$$

where x is *not free* in the resulting sequent.

Usage example:

```
have(B ⊢ P(y)) by ...
thenHave(B ⊢ ∀(x, P(x))) by RightForall
```

LeftForall. Introduce a universal quantifier on the left.

$$\frac{\Gamma, P(t) \vdash \Phi}{\Gamma, \forall x. P(x) \vdash \Phi} \text{LeftForall}$$

where t is any term.

Usage example:

```
have((B, P(t)) ⊢ R(t)) by ...
thenHave((B, ∀(x, P(x))) ⊢ R(t)) by LeftForall
```



Scratch area

End